

CUB3D — GUIA GERAL DE SPRINTS

Documentação Técnica de Arquitetura e Parsing • Edição P&B Econômica

SPRINT 0 • CORE ARCHITECTURE E INICIALIZAÇÃO			
ETAPA 00 — Core Architecture e Inicialização			
Dificuldade:	★★★★☆	Próxima Etapa:	01_parsing.md

OBJETIVO

Implementar a infraestrutura central: definir estruturas (`t_game`, `t_map`, `t_player`, `t_texture`, `t_ray`), sistema de inicialização, gerenciamento de memória, erros centralizados e destruição segura de recursos. **(Sem parsing, renderização ou raycasting).**

ARQUIVOS ENVOLVIDOS

`src/game/`: `init_game.c`, `destroy_game.c`, `init_player.c`
`src/utlis/`: `cleanup.c`, `errors.c`
`inc/`: `cub3D.h`, `types.h`

RESULTADO ESPERADO

```
main() -> init_game() -> game pronto -> ... ->
destroy_game() -> cleanup total
```

ORDEM DE IMPLEMENTAÇÃO

Parte 1 — Estruturas Principais & Parte 2 — `init_game()`

Definir em `types.h` e `cub3D.h` as structs centrais. Em `init_game.c`, alocar o jogo, zerar a memória e inicializar sub-estruturas (mapa, player, texturas).

Parte 3 — Valores Padrão & Parte 4 — Sistema de Erros

Atribuir `NULL` a ponteiros e zerar flags/dimensões. Em `errors.c`, centralizar saídas de erro no formato `"Error [Mensagem]"` via `error_exit()`.

Parte 5 — Cleanup & Parte 6 — `destroy_game()`

Em `cleanup.c`, criar funções reutilizáveis como `free_matrix()`. Em `destroy_game.c`, liberar recursos na ordem inversa: loop -> imagens -> texturas -> janela -> MLX -> mapa -> player -> game.

Parte 7 — Tratamento de Falhas & Parte 8 — Convenções

Garantir que falhas parciais em `malloc/open` acionem limpeza limpa. Manter funções pequenas, sem variáveis globais, com arquivos organizados por responsabilidade.

CHECKLIST & TESTES

- [] Estruturas `t_game`, `t_map`, `t_player`, `t_texture`, `t_ray`
- [] `init_game()` e sub-inicializadores funcionais
- [] Mensagens de erro centralizadas ("Error ...")
- [] `destroy_game()` e `cleanup()` sem vazamentos
- [] Norminette em `src/game`, `src/utlis`, `inc`
- [] Valgrind aprovado sem leaks/invalid reads

CASOS CRÍTICOS & DOD

Atenção: Tratar falha parcial liberando o alocado. Evitar double-free (funções de liberação devem aceitar `NULL`).

DoD: Infraestrutura base de memória, erros e destruição operacionais, pronta para receber o Parsing.

CUB3D — GUIA GERAL DE SPRINTS

Documentação Técnica de Arquitetura e Parsing • Edição P&B Econômica

SPRINT 1 • PERÍODO: 29/07 → 02/08 (PARTE 1)

ETAPA 01 — Parsing do Arquivo .cub (Pipeline & Elementos)

Dificuldade:

★★★★☆

Próxima Etapa:

02_validation.md

OBJETIVO

Implementar o pipeline para abrir, ler e validar a estrutura básica do arquivo `.cub`, extraíndo caminhos de texturas, cores de teto/chão em RGB e preenchendo `t_game`. (Sem renderização).

ARQUIVOS ENVOLVIDOS

src/parsing/: `path_validation.c`, `textures_parsing.c`, `color_parsing.c`, `map_parsing.c`, `map_building.c`, `parsing_utils.c`, `map_utils.c`, `grid_utils.c`
include/: `cub3D.h`, `types.h`

FLUXO DO PARSING

`main()` -> `validate args` -> `check ext` -> `open/read` -> `textures` -> `colors` -> `map` -> `build grid` -> `find player`

ORDEM DE IMPLEMENTAÇÃO (PARTES 1 A 4)

- Parte 1 — Validação do Arquivo (`path_validation.c`)

Validar extensão `.cub`, caminhos vazios, permissão de leitura, diretórios e inexistência via `check_file_extension()` e `validate_path()`.
- Parte 2 — Leitura do Arquivo (`map_parsing.c` / `parsing_utils.c`)

Ler o arquivo linha a linha via GNL em `read_file()`, armazenando o conteúdo bruto em uma matriz de strings para processamento.
- Parte 3 — Parsing das Texturas (`textures_parsing.c`)

Identificar e extrair os identificadores `NO`, `SO`, `WE`, `EA`. Garantir exatamente 1 ocorrência de cada. Rejeitar duplicadas ou caminhos inválidos.
- Parte 4 — Parsing das Cores (`color_parsing.c`)

Extrair identificadores `F` (chão) e `C` (teto). Converter string RGB `"R,G,B"` (com `0 <= valor <= 255`) para hexadecimal `0xRRGGBB` via `parse_color()`.

Testes de Erro em Cores/Texturas: Rejeitar `300`, `0`, `0`, `-1`, `0`, `0`, valores incompletos (`255`, `255`), caracteres não numéricos e texturas duplicadas.

CUB3D — GUIA GERAL DE SPRINTS

Documentação Técnica de Arquitetura e Parsing • Edição P&B Econômica

SPRINT 1 • PERÍODO: 29/07 → 02/08 (PARTE 2)

ETAPA 01 — Parsing do Arquivo .cub (Grid & Checklist)

ORDEM DE IMPLEMENTAÇÃO (PARTES 5 A 7)

- Parte 5 — Parsing do Mapa (map_parsing.c)

Detectar o início da seção do mapa no arquivo e isolar as linhas correspondentes na matriz de caracteres (`char **grid`).
- Parte 6 — Construção do Grid (map_building.c / grid_utils.c)

Normalizar todas as linhas do mapa para manter largura consistente. Preencher linhas menores conforme a estratégia do projeto.
- Parte 7 — Localizar o Jogador (map_utils.c)

Varrer a matriz em busca dos caracteres `N`, `S`, `E` ou `W`. Salvar `player.x`, `player.y` e a direção inicial. Rejeitar se houver 0 ou mais de 1 player.

CHECKLIST & CRITÉRIOS DE CONCLUSÃO

CHECKLIST

- ☐ Extensão .cub validada e arquivo lido
- ☐ Texturas NO, SO, WE, EA armazenadas
- ☐ Cores F e C convertidas para Hexadecimal
- ☐ Mapa isolado e grid normalizado
- ☐ Player localizado com direção inicial armazenada
- ☐ Testes de arquivos inválidos (bad_extension, duplicate_texture, bad_rgb, double_player) ok

DEFINITION OF DONE

DoD Código & Funcional: Abre arquivos válidos e rejeita inválidos, preenche `t_game` totalmente, passa na Norminette, 0 leaks/segfaults, pronto para Validation.

CUB3D — GUIA GERAL DE SPRINTS

Documentação Técnica de Arquitetura e Parsing • Edição P&B Econômica

SPRINT 2 • PERÍODO: 02/08 → 05/08 (PARTE 1)

ETAPA 02 — Validation do Arquivo .cub (Regras & Bordas)

Dificuldade:

★★★★★

Próxima Etapa:

03_mlx.md

OBJETIVO

Garantir que o mapa atende **todas as regras do subject** sem modificar seus dados. Somente mapas 100% válidos avançam para o módulo de renderização.

ARQUIVOS ENVOLVIDOS

src/parsing/: `map_validation.c`, `validation_utils.c`

REGRAS DO SUBJECT

1 jogador exato; apenas caracteres `[0, 1, N, S, E, W, ' ']`; totalmente fechado por paredes; sem vazamentos para o exterior.

FLUXO DA VALIDAÇÃO

grid -> dimensões -> caracteres -> player -> bordas -> flood fill -> mapa aprovado

ORDEM DE IMPLEMENTAÇÃO (PARTES 1 A 4)

Parte 1 — Dimensões do Mapa (validation_utils.c)

Validar largura e altura sobre o grid normalizado via `get_map_width()`, `get_map_height()` e `validate_dimensions()`.

Parte 2 — Caracteres Permitidos (map_validation.c)

Verificar se existem apenas caracteres válidos (`[0, 1, N, S, E, W, espaço]`). Rejeitar qualquer outro símbolo (ex: `[A, B, 2, #, @]`).

Parte 3 — Jogador Único

Garantir a presença de exatamente 1 jogador. Rejeitar mapas sem jogador ou com múltiplos jogadores (ex: `[NN]`, `[NS]`).

Parte 4 — Bordas do Mapa

Garantir que nenhuma área jogável (`[ '0' ]` ou Player) toque o limite externo do grid sem proteção de paredes (`[ '1' ]`).

Tratamento de Espaços: Espaços representam área externa. Nunca podem ter contato direto com áreas jogáveis (`'0'` /Player), pois isso constitui um vazamento de mapa.

CUB3D — GUIA GERAL DE SPRINTS

Documentação Técnica de Arquitetura e Parsing • Edição P&B Econômica

SPRINT 2 • PERÍODO: 02/08 → 05/08 (PARTE 2)

ETAPA 02 — Validation do Arquivo .cub (Flood Fill & DoD)

ORDEM DE IMPLEMENTAÇÃO (PARTES 5 A 7)

- Parte 5 — Tratamento de Espaços

Validar que espaços internos ou externos não criam comunicação com caminhos navegáveis do mapa via `validate_spaces()`.
- Parte 6 — Algoritmo Flood Fill (`map_validation.c`)

Teste definitivo para detectar vazamentos. A partir da posição do player em uma cópia do mapa, percorrer recursivamente/iterativamente em N, S, L, O. Se alcançar o espaço externo ou o limite da matriz, o mapa é inválido.
- Parte 7 — Mensagens de Erro Padronizadas

Exibir mensagens claras iniciadas com `Error` (ex: `"Error Map is not closed"`, `"Error Multiple players found"`) em `error_exit()`.

CHECKLIST & CRITÉRIOS DE CONCLUSÃO

CHECKLIST

- ☐ Dimensões do grid e caracteres validados
- ☐ Exatamente 1 player presente
- ☐ Bordas e espaços checados contra vazamentos
- ☐ Flood fill executado com sucesso sem vazamentos
- ☐ Mensagens de erro padronizadas com "Error"
- ☐ Norminette & Valgrind aprovados

DEFINITION OF DONE

DoD: Rejeita mapas abertos, com múltiplos players ou caracteres inválidos. Parser + Validation 100% integrados, sem leaks, pronto para MLX42.